

High Level Design (HLD) — Online Medical Consultation Platform

1. System Overview

The Online Medical Consultation Platform is a cloud-native telemedicine solution that enables patients to book appointments, consult doctors via video calls, make secure payments, and receive realtime notifications.

The system is designed for:

- Scalability
 - High Availability
 - Realtime Synchronization
 - Secure Transactions
 - Collision-Free Appointment Booking
-

2. High Level Architecture

Client Layer

|
--- Patient Web App (React)
--- Doctor Dashboard
--- Admin Dashboard

|
▼
API Gateway / Backend Layer

|
--- Authentication Service
--- Appointment Service
--- Doctor Assignment Engine
--- Payment Service
--- Notification Service
--- Video Consultation Service

|
▼
Database Layer

|
--- Firebase Firestore
--- Realtime Listeners
--- Distributed Locks Collection

|
▼
External Integrations

|

--- Firebase Auth
--- Razorpay
--- Jitsi Meet
--- Email/Push Notification Services

3. Core Modules

Module	Responsibility
Authentication Service	User login, session validation, RBAC
Appointment Service	Booking management and scheduling
Doctor Assignment Engine	Smart doctor allocation
Slot Locking Engine	Collision prevention
Payment Service	Razorpay integration
Video Consultation Service	Jitsi room management
Notification Service	Email/push reminders
Realtime Sync Service	Firestore listeners
Admin Service	Doctor approvals and monitoring

4. Functional Requirements

Patient Features

- Register/Login
- Fill medical questionnaire
- Book appointments
- Online payment
- Video consultation
- View appointment history

Doctor Features

- Doctor onboarding
- Availability management
- Consultation handling
- Appointment tracking

Admin Features

- Doctor approval

- Monitoring bookings
 - Analytics dashboard
-

5. Non-Functional Requirements

Requirement	Description
Scalability	Horizontal API scaling
Reliability	Transaction rollback support
Availability	Cloud Run auto scaling
Security	JWT + Firebase Auth
Consistency	Atomic Firestore transactions
Performance	Low latency realtime updates

6. Scalability Strategy

- Stateless backend deployment
 - Auto-scaling using Cloud Run
 - Firestore distributed architecture
 - Event-driven notifications
 - Realtime listeners for live updates
-

7. Security Design

- Firebase Authentication
 - JWT Token Validation
 - Role-Based Access Control
 - Firestore Security Rules
 - Secure Payment Verification
 - Protected Consultation Sessions
-

8. Reliability & Fault Tolerance

- Distributed lock handling
- Transaction rollback support
- Stale lock cleanup

- Retry mechanisms
 - Multi-device synchronization
 - Doctor failover assignment
-

9. Database Design Overview

Collections

users

Stores patient and admin details.

doctors

Stores doctor profile and availability.

questionnaires

Stores patient medical history.

appointments

Stores booking details.

locks

Stores temporary slot locks.

chat_messages

Stores realtime consultation chats.

10. Realtime Architecture

Realtime synchronization is implemented using Firestore onSnapshot listeners.

Updates include:

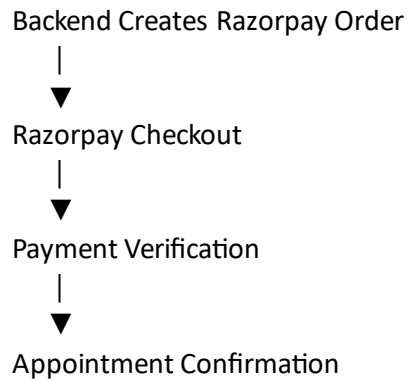
- Slot availability
 - Booking confirmations
 - Notification delivery
 - Chat synchronization
 - Consultation status
-

11. Payment Architecture

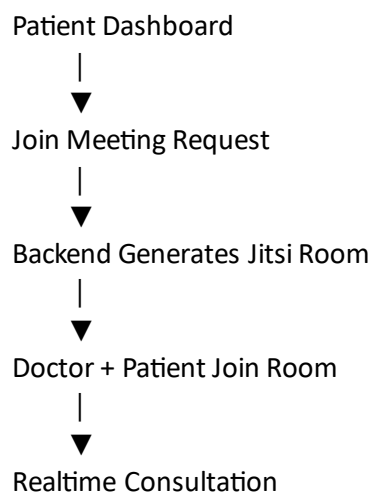
Patient

|





12. Video Consultation Architecture



13. Deployment Architecture

Layer	Technology
Frontend	React + TypeScript
Backend	Node.js + Express
Hosting	Google Cloud Run
Database	Firebase Firestore
Auth	Firebase Authentication
Payments	Razorpay
Video Calls	Jitsi Meet

14. High Level Workflow

Patient Login



Questionnaire Submission



Slot Selection



Doctor Assignment



Slot Lock Creation



Payment Processing



Booking Confirmation



Video Consultation



Consultation Completion